Everett Pohl, Dev Patel, Nikhil Vundela, Daniel Mendoza
ITIS 3200 001, Professor Xiang
Fall 2026, Team 3

**Course Project Proposal for SQL Injection Remediation**

For our project, we decided to look at SQL Injection Remediation for a Given Vulnerable Query Layer in the domain of Web Security. SQL Injection is a form of code injection that is very commonly used to attack websites. Hackers use it to inject malicious code into user input fields, allowing them access to sensitive data within a site's user database. Hackers typically perform this exploit by appending extra conditions to usually expected inputs, allowing them to receive not just the intended response, but also possibly the rest or a portion of a database. These attacks can leak sensitive data like usernames and passwords, or can allow hackers to modify or delete entire databases, wreaking havoc on a website.

The attacker is an outside user who is not logged in and can only type into the login and search fields. They cannot reach the server or database directly, and they have no valid password. We focus on:

1. login bypass, such as typing ' OR '1'='1' -- to get in without a password
2. data theft, such as using UNION in the search box to see other users' records;
3. data changes, such as deleting or editing records, which is possible because the database account has more permissions than the app needs.

These put the privacy of user data and the safety of the login at risk.

Our solution is to use parameterized queries for the login and search fields. These keep user input separate from the SQL commands, so an attacker cannot change what the query does by typing SQL into a field. We will also limit the database account's permissions to only what the app needs, which helps reduce the damage if something is missed. We will test the same attack on both versions to show that it works before the fix and fails afterward, while regular logins and searches still work.

The broken state we aim to solve is the SQL query layer. While this layer is necessary to access database records, it also provides the user the ability to alter the database, something we do not want. We aim to enforce these security principles:

- Authentication: Stop attackers from using fake/nonexistent login credentials to perform queries by requiring a secure login prior to executing queries.
- Least Privilege: Limit account permissions to ensure users cannot alter the database.
- Defense in Depth: The database will now be protected by two layers (parameter authentication + restricted query engine), making it harder to compromise.